Python Operators

Operators in Python are special symbols or keywords that perform operations on variables and values

---

1. Arithmetic Operators

Arithmetic operators are used for basic mathematical calculations.

Operator	Symbol	Example	Explanation
Addition	`+`	`5 + 3 = 8`	Adds two numbers
Subtraction	`-`	`10 - 4 = 6`	Subtracts the second number from the first
Multiplication	`*`	`6 * 2 = 12`	Multiplies two numbers
Division	`/`	`9 / 2 = 4.5`	Divides two numbers (gives float)
Floor Division	`//`	`9 // 2 = 4`	Divides and returns the integer part
Modulus (Remainder)	`%`	`10 % 3 = 1`	Returns the remainder of the division
Exponentiation	`**`	`2 ** 3 = 8`	Raises the first number to the power of the second

Example:

```python
a = 10
b = 3
print(a + b)   # Output: 13
print(a - b)   # Output: 7
print(a * b)   # Output: 30
print(a / b)   # Output: 3.3333
print(a // b)  # Output: 3
print(a % b)   # Output: 1
print(a ** b)  # Output: 1000
```

2. Comparison Operators

Comparison operators compare two values and return **True** or **False**.

Operator	Symbol	Example	Explanation
Equal to	`==`	`5 == 5` → `True`	Checks if both values are equal
Not Equal to	`!=`	`5 != 3` → `True`	Checks if values are not equal
Greater than	`>`	`7 > 3` → `True`	Checks if the first value is greater
Less than	`<`	`3 < 7` → `True`	Checks if the first value is smaller
Greater than or Equal to	`>=`	`5 >= 5` → `True`	Checks if the first value is greater or equal
Less than or Equal to	`<=`	`3 <= 5` → `True`	Checks if the first value is smaller or equal

Example:

Python

```
x = 10
y = 5

print(x == y)   # Output: False
print(x != y)   # Output: True
print(x > y)    # Output: True
print(x < y)    # Output: False
print(x >= 10)  # Output: True
print(y <= 5)   # Output: True
```

3. Assignment Operators

Assignment operators are used to assign or update the value of a variable.

Operator	Symbol	Example	Equivalent To
Assign	=	x = 5	Assigns value to variable
Add & Assign	+=	x += 3	x = x + 3
Subtract & Assign	-=	x -= 2	x = x - 2
Multiply & Assign	*=	x *= 4	x = x * 4
Divide & Assign	/=	x /= 2	x = x / 2
Floor Divide & Assign	//=	x //= 3	x = x // 3
Modulus & Assign	%=	x %= 2	x = x % 2
Exponentiate & Assign	**=	x **= 3	x = x ** 3
Bitwise AND Assign	&=	x &= 3	x = x & 3
Bitwise OR Assign	|=	x |= 3	x = x | 3
Bitwise XOR Assign	^=	x ^= 3	x = x ^ 3

Right Shift Assign	>>=	x >>= 2	x = x >> 2
Left Shift Assign	<<=	x <<= 2	x = x << 2
Walrus Assignment	:=	print(x := 10)	Assigns value inside expression

Example:

```python
x = 10
x += 5    # x = x + 5 → 15
x *= 2    # x = x * 2 → 30
x -= 10   # x = x - 10 → 20
print(x)  # Output: 20
```

4. Logical Operators (Using Logic Gates)

Logical operators are used to combine multiple conditions. They work similarly to **AND**, **OR**, and **NOT** gates in digital electronics.

Operator	Symbol	Example	Explanation
AND	`and`	`(x > 5 and y < 15)`	Returns **True** if **both** conditions are true
OR	`or`	`(x < 5 or y > 15)`	Returns **True** if **at least one** condition is true
NOT	`not`	`not (x > 5)`	Reverses the condition (True becomes False)

AND Gate Truth Table

A	B	A and B
True	True	True
True	False	False
False	True	False

False	False	False

OR Gate Truth Table

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

NOT Gate Truth Table

A	not A
True	False
False	True

Example:

```python
x = 10
y = 20

print(x > 5 and y < 30)  # Output: True (Both conditions are True)
print(x > 15 or y < 30)  # Output: True (At least one condition is True)
print(not (x > 5))       # Output: False (Reverses the True condition)
```

5. Membership Operators

Membership operators check if a value exists within a sequence (like a list, string, or tuple).

Operator	Symbol	Example	Explanation
In	`in`	`"a" in "apple"`	Returns **True** if the value exists in the sequence

Not In	`not in`	`"x" not in "apple"`	Returns **True** if the value does **NOT** exist in the sequence

Example:

```python
fruits = ["apple", "banana", "cherry"]

print("apple" in fruits)      # Output: True
print("grape" not in fruits)  # Output: True
```

6. Identity Operators

Identity operators check whether two variables refer to the **same object** in memory.

Operator	Symbol	Example	Explanation
Is	`is`	`x is y`	Returns **True** if both refer to the same object
Is Not	`is not`	`x is not y`	Returns **True** if they refer to different objects

Example:

```python
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a is b)      # Output: True (Both refer to the same object)
print(a is c)      # Output: False (Different objects with the same content)
print(a is not c)  # Output: True
```

7. Bitwise Operators

Bitwise operators perform operations on the **binary representation** of numbers (0 and 1).

Operator	Symbol	Example	Explanation
AND	`&`	`5 & 3 = 1`	Performs bitwise AND

OR	|	5 | 3 = 7	Performs bitwise OR
XOR	^	`5 ^ 3 = 6`	Performs bitwise XOR
NOT	~	`~5 = -6`	Performs bitwise NOT
Left Shift	<<	`5 << 1 = 10`	Shifts bits to the left (multiplies by 2)
Right Shift	>>	`5 >> 1 = 2`	Shifts bits to the right (divides by 2)

Logic Gates Truth Table (for bits 0 and 1)

A	B	A AND B	A OR B	A XOR B	NOT A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Binary Representation Example:

Python

```
x = 5  # Binary: 0101
y = 3  # Binary: 0011

print(x & y)   # Output: 1  (Binary: 0001 → AND operation)
print(x | y)   # Output: 7  (Binary: 0111 → OR operation)
print(x ^ y)   # Output: 6  (Binary: 0110 → XOR operation)
print(~x)      # Output: -6 (Binary: Inverts bits of 5)
print(x << 1)  # Output: 10 (Binary: 1010 → Shifts left by 1)
print(x >> 1)  # Output: 2  (Binary: 0010 → Shifts right by 1)
```

Input Formatting

Python's `input()` function is used to take input from the user during program execution.

By default:

```python
input()
```

always returns a string datatype.

1. String Input

Use case:

- Name
- Email
- City

```python
name = input("Enter your name: ")
print(name)
Enter your name: Ravi
Ravi
```

2. Integer Input

Use case:

- Age
- Quantity
- Marks

```python
age = int(input("Enter age: "))
print(age)

Enter age: 25
25
```

3. Float Input

Use case:

- Price
- Rating
- Temperature

```python
price = float(input("Enter price: "))
print(price)

Enter price: 99.5
99.5
```

4. List of Strings

Use case:

- Names
- Subjects
- Cities

```python
names = input("Enter names: ").split()
print(names)

Enter names: Ravi Teja Ankit
['Ravi', 'Teja', 'Ankit']
```

5. Tuple of Strings

```python
names = tuple(input("Enter names: ").split())
print(names)

Enter names: Ravi Teja Ankit
('Ravi', 'Teja', 'Ankit')
```

6. Set of Strings

```python
names = set(input("Enter names: ").split())
print(names)

Enter names: Ravi Teja Ravi
{'Ravi', 'Teja'}
```

Duplicates are removed in sets.

7. List of Integers

```python
numbers = list(map(int, input("Enter numbers: ").split()))
print(numbers)

Enter numbers: 10 20 30
[10, 20, 30]
```

8. Tuple of Integers

```python
numbers = tuple(map(int, input("Enter numbers: ").split()))
print(numbers)
Enter numbers: 10 20 30
(10, 20, 30)
```

9. Set of Integers

```python
numbers = set(map(int, input("Enter numbers: ").split()))
print(numbers)

Enter numbers: 10 20 20 30
{10, 20, 30}
```

10. List of Floats

```python
values = list(map(float, input("Enter float values: ").split()))
print(values)

Enter float values: 10.5 20.8 30.1
[10.5, 20.8, 30.1]
```

11. Tuple of Floats

```python
values = tuple(map(float, input("Enter float values: ").split()))
print(values)

Enter float values: 10.5 20.8 30.1
(10.5, 20.8, 30.1)
```

12. Set of Floats

```python
values = set(map(float, input("Enter float values: ").split()))
print(values)

Enter float values: 10.5 20.8 20.8 30.1
{10.5, 20.8, 30.1}
```

13. Input using eval()

`eval()` can take:

- dict
- list
- tuple
- set
- int
- float
- string
- boolean

and converts entered Python expressions into actual datatypes.

Dictionary Input

```python
data = eval(input("Enter dictionary: "))
print(data)
print(type(data))

Enter dictionary: {'name':'Ravi','age':25}
{'name': 'Ravi', 'age': 25}
<class 'dict'>
```

List Input using eval()

```python
data = eval(input("Enter list: "))
print(data)
print(type(data))

Enter list: [10,20,30]
[10, 20, 30]
<class 'list'>
```

Tuple Input using eval()

```python
data = eval(input("Enter tuple: "))
print(data)
print(type(data))

Enter tuple: (10,20,30)
(10, 20, 30)
<class 'tuple'>
```

Set Input using eval()

```python
data = eval(input("Enter set: "))
print(data)
print(type(data))

Enter set: {10,20,30}
{10, 20, 30}
<class 'set'>
```

Integer Input using eval()

```python
data = eval(input("Enter integer: "))
print(data)
print(type(data))

Enter integer: 100
100
<class 'int'>
```

Float Input using eval()

```python
data = eval(input("Enter float: "))
print(data)
print(type(data))

Enter float: 55.5
55.5
<class 'float'>
```

Boolean Input using eval()

```Python
data = eval(input("Enter boolean: "))
print(data)
print(type(data))

Enter boolean: True
True
<class 'bool'>
```

Warning about eval()

```Python
eval()
```

executes the entered input as Python code.

Use it only for:

- Trusted input
- Learning purposes
- Controlled environments

Avoid using `eval()` with unknown user input because it may create security risks.

Multiple Inputs with Unpacking

```Python
username, password = input("Enter username and password:
").split()
print(username)
print(password)

Enter username and password: admin admin123
admin
admin123
```

Multiple Integer Inputs

```python
a, b = map(int, input("Enter two numbers: ").split())
print(a)
print(b)

Enter two numbers: 10 20
10
20
```

Summary Table

Input Type	Example	Code
String	Ravi	input()
Integer	25	int(input())
Float	55.5	float(input())
List of str	Ravi Teja	input().split()
Tuple of str	Ravi Teja	tuple(input().split())
Set of str	Ravi Teja	set(input().split())
List of int	10 20 30	list(map(int, input().split()))
Tuple of int	10 20 30	tuple(map(int, input().split()))
Set of int	10 20 30	set(map(int, input().split()))
List of float	10.5 20.8	list(map(float, input().split()))
Tuple of float	10.5 20.8	tuple(map(float, input().split()))
Set of float	10.5 20.8	set(map(float, input().split()))
Dictionary	{'a':1}	eval(input())
Multiple Inputs	admin 123	a, b = input().split()

Output Formatting

Understanding the print() Function in Python

Before learning output formatting, we must understand the print() function.

`print()` is used to display output on the screen.

Basic Syntax

```python
print(object, sep=' ', end='\n')
```

Parameter	Description
object	Data to display
sep	Separator between multiple items
end	What to print at the end

1. Printing Text

```python
print("Hello World")
Hello World
```

2. Printing Multiple Values

```python
name = "Alice"
age = 25
print("Name:", name, "Age:", age)

Name: Alice Age: 25
```

Python automatically adds spaces between items.

3. Using sep Parameter

`sep` changes the separator between values.

```
Python
print("2026", "07", "15", sep="-")
2026-07-15
```

4. Using end Parameter

`end` changes what is printed at the end.

Default: `end = '\n'` means new line.

Example:

```
Python
print("Hello", end=" ")
print("World")
Hello World
```

Special Characters in Output

Special Character	Meaning
\n	New Line
\t	Tab Space
\	Backslash
'	Single Quote
"	Double Quote

New Line Character (\n)

```
Python
print("Line1\nLine2")
Line1
Line2
```

Tab Character (\t)

```python
print("Name:\tAlice")
Name:    Alice
```

OUTPUT FORMATTING METHODS

Python supports multiple ways to format output.

1. Using Commas in print()

Simplest method.

```python
name = "Alice"
age = 25
score = 95.5

print("Name:", name, "Age:", age, "Score:", score)

Name: Alice Age: 25 Score: 95.5
```

Advantages

- Easy for beginners
- Simple syntax

Disadvantages

- Less formatting control

2. Using Modulo (%) Formatting

Old C-style formatting method.

Format Specifiers

Specifier	Datatype
%s	String
%d	Integer
%f	Float

Python
```
name = "Bob"
age = 30
score = 88.75

print("Name: %s | Age: %d | Score: %.2f" % (name, age, score))

Name: Bob | Age: 30 | Score: 88.75
```

Advantages

- Better formatting control
- Supports decimal formatting

Disadvantages

- Old method
- Less readable

3. Using f-Strings

Most modern and recommended method.

Python
```
name = "Charlie"
age = 28
score = 92.389
print(f"Name: {name} | Age: {age} | Score: {score:.2f}")

Name: Charlie | Age: 28 | Score: 92.39
```

Advantages

- Easy to read
- Fast
- Modern
- Supports expressions directly

Disadvantages

- Works only in Python 3.6+

4. Using str.format() Method

```python
name = "Diana"
age = 22
score = 89.456
print("Name: {} | Age: {} | Score: {:.1f}".format(name, age,
score))

Name: Diana | Age: 22 | Score: 89.5
```

Positional Formatting

```python
print("Name: {0} Age: {1}".format("Ravi", 25))
Name: Ravi Age: 25
```

Keyword Formatting

```python
print("Name: {name} Age: {age}".format(name="Ravi", age=25))
Name: Ravi Age: 25
```

Advantages

- Flexible
- Powerful formatting support

Disadvantages

- Longer syntax compared to f-strings

NUMBER FORMATTING

Comma Separator

```python
amount = 1000000
print(f"{amount:,}")

1,000,000
```

Percentage Formatting

```python
value = 0.85
print(f"{value:.0%}") #85%
```